Iceberg Lake: Trace History and Traversable Provenance

> **Implemented, opt-in, CP only** — root README §3.5.4. `make cp-flink-reports-up ENABLE_LAKE=true`, optionally with `ENABLE_STATE_PROVENANCE=true`. Code: `IsotopeLakeJob`, `90_lake_catalog.fql`, `91_lake_state_provenance_sink.fql`, `95_lake_hops.fql`, `96_lake_state_provenance.fql`, `iceberg-rest.yaml`, `lake-query.sh`.

Table of Contents

- 1.0 What the lake adds
- 2.0 Architecture
  - 2.1 A second application, not more INSERTs
  - 2.2 Tables
  - 2.3 Catalog and storage
- 3.0 Running it
- 4.0 Querying it
- 5.0 Limits and deliberate omissions

1.0 What the lake adds

Everything else in this project analyzes in-flight data. The seven reports fold hops into 1-minute windows and keep only the aggregate; Prometheus keeps aggregates for as long as its retention allows; the raw isotope lives in Kafka for as long as the topic's retention allows. The lake keeps the rows:

Without the lake	With the lake
A new question needs a new Flink statement and a replay of Kafka	A new question is one SQL query over every hop ever recorded
Per-trace history ends at topic retention	Per-trace history is kept in object storage for as long as you want it
State-provenance ancestry is *queryable, not traversable* — Flink SQL has no recursive CTEs (state-provenance.md §6.0)	`WITH RECURSIVE` walks every version chain end to end

The last row is the reason the lake exists. `STATE_PROVENANCE` output was designed for it: append-only, with a content-addressed `version_id`, so it lands as a pure-append Iceberg table — no equality deletes, no merge-on-read, and a redelivered record is the same version rather than a new one.

2.0 Architecture

2.1 A second application, not more INSERTs

The reports run as one StatementSet in one CMF Application — one failure domain, where a missing sink topic already stops all seven. An Iceberg sink in that set would add a catalog and an object store to the list of things that can take the reports down.

So the lake is its own CMF Application, `isotope-lake`. It runs from the **same shadow JAR and the same `cmf://` artifact**, selected by `entryClass: IsotopeLakeJob` in `cmf-flink-lake-application.json`, and reads the Kafka topics on its own under its own consumer group (`isotope-lake`). The two applications share nothing at runtime: stop, break or redeploy the lake and the reports never notice.

It checkpoints every **60s** rather than the reports' 30s. Iceberg commits on each checkpoint, so the interval is both the lake's freshness and the size of the files each commit writes.

It **resumes from its consumer group's committed offsets** (`scan.startup.mode = group-offsets`, falling back to `earliest` for a group that has never committed), where every source in `00_source_table.fql` says `earliest-offset`. The applications deploy with `upgradeMode: stateless`, so a redeploy has no checkpoint to restore. The reports can afford to replay from the start, because it recomputes the same windows into a Kafka topic. An append-only lake table can't: the first version of this job re-copied every hop on each redeploy (48 hops became 96 rows). Offsets are committed when a checkpoint completes, which is the same moment Iceberg commits, but the two aren't atomic: a crash between them can drop or repeat at most one checkpoint's worth of rows on the next start. Proper exactly-once across redeploys would need `upgradeMode: savepoint` with checkpoint storage on RustFS, which is out of scope here.

2.2 Tables

Table	Source	Written when
`lake.isotope.hops`	the `isotope` view (`05_isotope_view.fql`), row for row	always, with `ENABLE_LAKE=true`
`lake.isotope.state_provenance`	the `isotope_state_provenance` **topic**, row for row	with `ENABLE_LAKE=true` `ENABLE_STATE_PROVENANCE=true`

`state_provenance` reads the topic rather than re-running the PTF. The reports application already owns `STATE_PROVENANCE` and its keyed state; a second copy would derive identical IDs (they are content-addressed) from twice the state for nothing. The topic is pinned to infinite retention, so the lake can always rebuild from its first record.

Both tables are unpartitioned: at demo volume a partition spec only multiplies the small files each checkpoint commits. At real volume, partition `hops` by `day(event_time)` from the catalog side — Flink DDL cannot express Iceberg transforms.

2.3 Catalog and storage

- **Catalog:** the Apache Iceberg REST fixture (`iceberg-rest.yaml`) — one pod, a SQLite catalog on a PVC. No auth server, no database pod, no bootstrap. The REST protocol is versioned separately from the Iceberg library, so the 1.10.x server serves the 1.11.x Flink runtime.
- **Storage:** the `isotope-lake` bucket on the RustFS instance CMF already uses for artifacts. Iceberg writes through its own `S3FileIO` (path-style), not the `s3-fs-hadoop` plugin.
- **Runtime:** `iceberg-flink-runtime-2.1` 1.11.0 (the first Iceberg release with a Flink 2.1 runtime), plus a pinned list of 45 JARs, `lake-jars.txt`, baked into the Flink image by `flink-sql-isotope.containerfile`. The list holds:
  - **The AWS SDK S3 client**, using the JDK's own HTTP stack (`http-client.type = urlconnection`). There's no Netty or Apache HTTP client, because the lake never calls them.
  - **The KMS and STS SDK modules.** Iceberg's `AwsProperties` can't initialize without them, although nothing here calls KMS or STS.
  - **A slice of Hadoop 3.5.0:** `hadoop-common`, `hadoop-hdfs-client`, `hadoop-auth` and the libraries `Configuration` and `UserGroupInformation` need. Iceberg's Flink catalog factory

builds a Hadoop `Configuration` even for a REST catalog, and Flink calls `UserGroupInformation` at startup once Hadoop is on the classpath. The `s3-fs-hadoop` plugin's copy is classloader-isolated from `/opt/flink/lib`.

The list replaces `iceberg-aws-bundle` and `hadoop-client-runtime`, which carried 1 critical and ~24 high CVEs in libraries the lake never loads. Nothing on the reports path loads any of these JARs. On the reports application Flink logs `Cannot create Hadoop Security Module ... No security module will be loaded`, which is harmless: this demo runs no Kerberos.

3.0 Running it

```
make cp-flink-reports-up ENABLE_LAKE=true                                  # hops
only
make cp-flink-reports-up ENABLE_LAKE=true ENABLE_STATE_PROVENANCE=true   # hops
+ version chains
```

`ENABLE_LAKE=true` runs `make lake-up` first (catalog + bucket), deploys the reports application as usual, then deploys `isotope-lake`. Drive traffic as in runbook-local.md §6.0; rows appear within one checkpoint (~60s).

Teardown: `make cp-flink-reports-down` deletes the lake application whether or not `ENABLE_LAKE` is set (so an earlier `up` is never stranded). `make lake-down` deletes the catalog, its PVC and the bucket's contents; `make cp-flink-down` runs it before `rustfs-down`.

> **Scheduling headroom.** The lake adds a JobManager (0.25 CPU) and a TaskManager (0.5 CPU). The Flink operator chart requests 2 full CPUs by default, which left no room on the 6-CPU minikube node — the lake TaskManager sat `Pending` with `Insufficient cpu`. `make flink-operator-install` now requests 500m for the operator (limit still 2).

4.0 Querying it

`scripts/lake-query.sh` reads the lake with **DuckDB on the host** (`brew install duckdb`) through two port-forwards — the catalog and RustFS — so querying costs the cluster nothing.

```
scripts/lake-query.sh              # chains: every entity's version chain,
walked with WITH RECURSIVE
scripts/lake-query.sh latency      # per-topic p50/p95/p99 over the lake's
whole history
scripts/lake-query.sh hops         # 20 most recent hops
scripts/lake-query.sh hops <TRACE> # every hop of one trace
scripts/lake-query.sh shell        # interactive DuckDB, lake attached as
`lake`
```

The `chains` walk starts at each **leaf** — a version no other version names as a parent — and follows `parents` until a version has none. It deduplicates first: the topic is at-least-once, and a duplicate `version_id` is by construction the same version.

5.0 Limits and deliberate omissions

- **CP only.** State provenance is CP only (state-provenance.md §5.0), so the half of the lake that matters most is too. On CCAF, Tableflow could materialize a flattened hops topic as Iceberg; that is not built.
- **Write-only from Flink.** The Hadoop slice covers what writing needs. Reading an Iceberg table back into Flink would also need Hadoop's MapReduce input classes, which aren't shipped. Reads go through DuckDB (`scripts/lake-query.sh`). Add `hadoop-mapreduce-client-core` to `lake-jars.txt` if a Flink job ever needs to read the lake.
- **Known CVEs that remain** (Trivy, CRITICAL/HIGH, when this was written):

Where	Findings	Why it isn't fixed here
`cp-flink:2.1.2-cp1` OS packages (openssl, util-linux, libxml2, pcre2, ...)	21 high	The base is a stripped UBI image with no `rpm`, `dnf` or `microdnf`, so it can't be patched in place. The newest 2.1 tag, `2.1.3-cp2`, fixes 4 of these but adds 22 Go stdlib findings (1 critical).
Shaded inside `iceberg-flink-runtime-2.1` 1.11.0 (httpcore5 5.4, Jackson 2.21.3)	5 high	Relocated inside Iceberg's own JAR. Only a newer Iceberg release can fix them, and 1.11.0 is the latest.
`apache/iceberg-rest-fixture:1.10.1`	1 critical, 29 high (Netty, Jackson, Jetty)	It's Iceberg's reference test fixture, fine for a single-user minikube demo and not for anything shared. A real deployment would run Polaris or Lakekeeper.

  The lake JAR list itself (`lake-jars.txt`) scans clean. Rescan it whenever it changes.

- **No table maintenance.** Each checkpoint commits one small file per table, and nothing compacts them or expires snapshots. Fine at demo volume; a real deployment schedules `rewrite_data_files` and `expire_snapshots`.
- **The seven report topics are not mirrored.** They are derivable from `hops` — which is the point of keeping the rows.
- **The lake inherits whatever its sources emit.** It copies the `isotope` view and the provenance topic verbatim, so a defect upstream lands in the lake as-is — but because `hops` keeps both `origin_ts_ms` and `event_time`, derived columns can always be recomputed there.